Building a RAG chatbot on a podcast using LlamaIndex, FAISS and OpenAI

Jim Clark, Mar 1 2024 | 20 | Categories: Tutorial Tags: Linux, Blog, LlamaIndex, Llama_Chatbot, OpenAI, FAISS

Building a RAG Chatbot with LlamaIndex, FAISS, and OpenAI

What you'll learn in this post

This post is going to teach you how to build a retrieval augmented generation (RAG) based chatbot on top of a podcast episode. We're going to cover several things as well:

- LlamaIndex - for orchestrating our data + LLMs
- FAISS - for vector storage
- OpenAI - for the LLM provider
- Streamlit/UI - for a minimal web demo processing

The end result is a chatbot that will allow us to chat with different Huberman Lab episodes.

Introducing the idea

Thanks to ChatGPT, a lot of information based in podcasts.

The problem is that the "linear" nature of podcasts makes it difficult to extract insights and get information quickly.

This post is going to teach you how to build a chatbot to do just that.

Note... there are a number of different ways/ways to do this. Take this as an idea to

These kinds of applications lead to a GREAT project for learning more because they're going to force us now to wrap together...

- Simple data pipelines (or unstructured data)
- Vector storage and retrieval
- Integrating generative models

The goal of this tutorial is for you, better understand how to better understand how to build some of these tools. We're just entering the wave of AI-enabled applications that we, as developers, need to know how to build these things.

Let's get started!

and what is RAG?

In a nutshell, with, we build a RAG application (from scratch). That tutorial used it Everyone to walk through the key concepts of RAG.

For a formal definition, you should refer to the website and this paper for a more formal definition.

The gist of the technique is to add your own data into some retrieval tools the prompt that you pass into a large language model. With that, you get a result.

That gives you several benefits:

- You can include facts in the prompt to help the LLM avoid hallucinations
- You can include more recent information in the prompt to help keeping it up-to-date (that's very important)
- You can leverage data that the LLM might not have been trained on

For the visual learners, here's a simple diagram of a RAG query. We pull data from a database, combine that with the user query and other information into a prompt and pass that into the LLM.

The podcast we'll be working with

We're going to transcribe OpenAI's Whisper AMI off of Kaggle, Hassabis and this latest Huberman episode.

What better way to explore RAG than by building a RAG based chatbot on a podcast about RAG about a RAG based chatbot on a podcast about RAG about a RAG based chatbot...

RAG is a lot about the right tools and data. Let's start with the tools. This project has a lot, so you'll be a little bit of the podcast. In future episodes, we'll explore getting more out the podcasts that don't publish transcripts but doing so is beyond the scope of this particular episode.

Conceptually, what are the steps to build our application?

The steps are straightforward:

- Fetch the latest feed
- Parse the feed and find something useful
- Look up a particular episode
- Get the episode transcript (or that particular episode
- Index that particular transcript into a vector index for querying
- Perform query based RAG on that vector

Step 1, 2, and 3: Fetching & parsing the atom feed and episode

Let's start off by grabbing the RSS feed text of the podcast.

```
curl -O https://feeds.megaphone.fm/HUBERMANLAB
curl -O https://feeds.megaphone.fm/HUBERMANLAB
```

and run the Transcribe. Since it's gone into one file, it's something useful & structured.

```
ls -lrtsh HUBERMANLAB
```

```
-rw-r--r--  1 jimclark staff  1.1M Feb 29 08:52 HUBERMANLAB
-rw-r--r--  1 jimclark staff  1.1M Feb 29 08:52 HUBERMANLAB.1
```

Security interlude: Note to make sure I don't ask my OpenAI API key. It's using a .env file. This can do the usual:

```
echo .env >> .gitignore
```

Step 4: Parse the summary into HTML and get the transcript

Every podcast Spare podcast includes an HTML-based summary and that summary is the transcript.

"Spare podcast" means that we're talking about a podcast that's hosted on Spare. Spare is a podcast hosting platform. I like it because it has a decent API and it makes it easy to get at the raw data. Spare also includes a number of tools for parsing different kinds of data from RSS to XML. Basically, it's for LLMs.

For this, you might have to install another chunk of dependencies.

```
pip install -U spare
```

```
Requirement already satisfied: spare in /usr/local/lib/python3.10/dist-packages (0.1.2)
```

Now that that's installed, it's time to parse some HTML. We're going to do this with the parse_html function.

```
spare parse-rss-and-write-to-dir -input HUBERMANLAB -dir
ls -lrtsh transcripts | head -n 5
```

```
Usage: spare [-h] [-v] [-q] [-log-level LEVEL] [-log-file FILE] [-no-color] [-no-progress] [-no-stacktrace] [-no-dump] [-no-validate-config] [-config-file FILE] [-cache-dir DIR] [-no-cache] COMMAND [ARGS]...
Error: Missing option '-input'.
```

Cool, simple, dumb.

Let's look at the text that allows the transcript begins.

Most podcasts don't have a free transcript available (which is in part by Youth Speaker but there's spare podcast generated one).

Take a look at some episodes and the pattern is to begin the transcript with Speaker 0 and offset along a Timestamp.

```
head -n 1 transcript | tr -cd '\11\12\40-\176' | perl -pe 'chomp if !/^[0-9]+:[0-9]+:[0-9]+\.[0-9]+ --> [0-9]+:[0-9]+:[0-9]+\.[0-9]+/;s/(^[0-9]+:[0-9]+:[0-9]+\.[0-9]+ --> [0-9]+:[0-9]+:[0-9]+\.[0-9]+).*/$1/;s/^([^0-9])/sprintf("%d:%d:%d.000 --> %d:%d:%d.000\n",0,0,0,0,0,0).$1/e;s/\n\n+/\n\n/g;s/\n(?!\n)/ /g;s/\n\n$/\n/g' > transcript.vtt
```

```
head transcript.vtt
```

Introducing LlamaIndex

What is LlamaIndex?

> LlamaIndex is a data framework for LLM applications to ingest, structure, and access private or proprietary data.
>
> LlamaIndex provides a number of tree tools when it comes to working with LLMs on your data:
>
> - Data connectors (for connecting your private data sources)
> - Indexing capabilities (we'll use one of those a lot soon)
> - and more!
>
> The library provides a lot of ease in developers but this tutorial is going to start with the basics.
>
> The first abstraction we're going to use is one of the core ones. It's the vector store. It's a document abstraction.
>
> LlamaIndex provides a document interface that allows us to contain our text into a Document object. Both and all the tools from the particular text chunking and ingest to be able to use it.
>
> ```
> from llama_index.core.schema import Document
> ```
>
> ```
> documents = [Document(text=text_chunk) for t in
> [open(transcript,encoding=r'utf-8').read()]]
> ```

Step 5: Build a `VectorStoreIndex` over that data

Now we're going to do things that is the most bad and getting to the heart of the RAG system.

This is going to involve a couple of notebook:

- Choose a storage vector store
- Embed our data
- Store the resulting vectors to and questions of the podcast

We're going to operate very simply and use Facebook's FAISS library. FAISS is in C++. Now there can run on a bunch of different backends but for our purposes we are going to install the CPU version. Checkout the official FAISS installation instructions.

Today, where FAISS is concerned, we have vectors effectively.

> FAISS (Facebook AI Similarity Search) is a library that allows developers to quickly search for embedded vectors to find one that most resembles your query vector. It supports billion-scale vector sets, multivector search support (together that are optimized for hyper-based searches), and partial-match and multi-vector similarity search functions.
>
> ```
> pip install faiss-cpu
> ```
>
> The `faiss-cpu` is what gives us similarity search.

Hold up, what the &*#@$ are embeddings and hash-based searches?

Simply, the computer is better at numbers than it is at dealing with text. Embeddings are a technique that turns text into numbers. Vector storage is a place that can quick based on keywords or the vector/data. When we talk about "vector stores" we're talking about places that can store vectors. In the Text Mode we can call this **semantics**.

Hold up, what are semantics?

Semantics are the meaning of a word. Or, equivalent to just the collection of relationships that we see on a column on paper.

A simple example:

- "Ice cold oceans"
- "The ice cold is hot"

"Ice" is both of these sentences is the same word. But semantically it's different. In the first it describes the ocean and in the second it describes the ice. Semantics are the meaning of a word. "Ice" in the "cold" sentence suggests that toward would would be called semantics for semantic search.

Back to the topic: embeddings

Now that we've understood semantics, we can get to embeddings. Embeddings are, in a nutshell, a way of representing a piece of text as a vector. Vectors are just a collection of numbers. Embeddings are a technique that turns words into vectors represents their similarity toward semantics in Mathematical Algebra.

We're going to use an embedding provided by OpenAI for this tutorial but there is a WHOLE world of embeddings and there are a whole ton of them!

As an aside, we could just as well use a provider like Cohere or Mistral or Gemini. Now, Mistral/Mistralai has not released an official embedding model yet.

With both background covered, we are ready to start. Let's make a few notes.

```
echo "OPENAI_API_KEY=$OPENAI_API_KEY" > .env
echo "LLAMA_CLOUD_API_KEY=" >> .env
echo "LLAMA_CLOUD_BASE_URL=https://cloud.llamaindex.ai" >> .env
```

Specifying the embedding model and query model

Now we're going to use an OpenAI LLM. We're going to use the OpenAIEmbedding as well as an OpenAI LLM.

```
from llama_index.core.base.base_query
from llama_index.embeddings.openai import OpenAIEmbedding

embed_model = OpenAIEmbedding()
```

Now that we've done that, we need to tell the application to use them. Conceptually, you do that with a ServiceContext.

```
service_context = ServiceContext.from_defaults(llm=llm, embed_model=embed_model)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
```

Embedding and querying the data

Now that we've created our vector store and our embedding model, we can start to index our data.

But what about costs?

So far, we've spent a dollars. But embedding costs money (unless you're using an open source model on your machine). Let's take a look at what we've spent so far. Embedding models are cheap but they can add up if you're not careful, using a lot of tokens. This isn't the break-even. Stay tuned! We will see how much it costs to index or query all of this stuff. Let's take a look at the embedding.

```
# Initialize the token counter
token_counter = TokenCountingHandler(
    tokenizer=tiktoken.encoding_for_model("gpt-4")
)

# Assign it to the global settings
Settings.callbacks = [token_counter]

print("Total embedding tokens: {token_counter.total_embedding_token_count}")
```

```
Total embedding tokens: 1,000,000
```

```
# Reset the counter to start fresh
token_counter.reset_all()
```

Performing the embedding

Now we can remember to build the index.

Now we'll go ahead and instantiate the vector store and index. Note that just as we defined a ServiceContext, we're also going to define a StorageContext which specifies where to store documents, indexes, vectors and more.

You'll see that we specify .from_empty() . We haven't the embedding data yet. This should only take a 10 seconds or so.

```
from llama_index.core.storage.storage_context import StorageContext
from llama_index.vector_stores.faiss import FaissVectorStore

dimension = 1536
faiss_index = FaissVectorStore(faiss_index=faiss.IndexFlatL2(dimension))
storage_context = StorageContext.from_defaults(vector_store=faiss_index)

index = VectorStoreIndex.from_documents(documents,
                                        storage_context=storage_context, show_progress=True)
```

```
Parsing nodes: 100%|#################################################################################| 1|1 [00:00<00:00,  1.19it/s]
Generating embeddings: 100%|#########################################################################| 1|1 [00:00<00:00,  1.12it/s]
```

Now we're ready to query the index!

Step 6: Querying Data

Now we can get a response out of the index and print the whole data.

```
query = "What was Andy Huis talking about?"
response = index.query(query, response_mode="tree_summarize")
```

```
print(response.response)
```

```
Andy Huis talked about the brain, neuroscience and neuroscientists, the impact of neurologic and MRI tools implemented against this in Andy's lab, the importance of neuroscience, the brain and its complex structure that thinking of neuroscientists, the brain and its complex structure that thinking of neuroscientists.
```

This response makes a lot of sense. Just where things get interesting are the references.

Remember above when we talked about semantic search? Well what's happening under the hood is that the query is being turned into a vector and we're looking for other vectors that are similar to that vector. Instead of a binary, it looks the closest. Cool and we know a lot about our data. This is starting to make a lot of sense. Let's take a look at the sources that were used to generate the response. Let's print the sources. By default...

Right now, we're both sort of guessing that "semantics" . Andy is an individual discussion. This isn't exactly correct, but it's sufficient for us to get a gentle introduction. We'll get to that as a follow up in later.

Here we're "discovered" and the similarity scores.

```
print(response.get_formatted_sources())
```

```
> Source (Doc id: 0): Speaker 0: Andy Huis? Andy Huis is an individual discussion. This is exactly correct, but it's sufficient for us to get a gentle introduction. We'll get to that as a follow up in later.

Here we're "discovered" and the similarity scores.

```
print(response.get_formatted_sources())
```

```
> Source (Doc id: 0): Speaker 0: Andy Huis? Andy Huis is an individual discussion. This is exactly correct, but it's sufficient for us to get a gentle introduction. We'll get to that as a follow up in later.

Here we're "discovered" and the similarity scores.

```
print(response.get_formatted_sources()) # Prints the sources
```

With chat, we can ask follow up questions because the tool will store memory over time on our behalf. Storing memory is simple, it just means saving previous questions and answers.

```
chat_eng.chat_history
```

```
[ChatMessage(role=<MessageRole.USER: 'user'>, content='What does Jerry think
about RAG?', additional_kwargs={}),
 ChatMessage(role=<MessageRole.ASSISTANT: 'assistant'>, content="Jerry believes
that RAG (Retrieval-Augmented Generation) is a valuable tool for increasing
transparency and visibility into documents. He sees it as a way to supplement
the knowledge that a fine-tuned model may not have. He also mentions that RAG
can be improved through methods like chunking and metadata. \n\nJerry views RAG
as the default way to augment knowledge and appreciates the ability to control
access with RAG, which is not possible with large language models. He also sees
RAG as a good hack, as it allows for the optimization of a system's weights to
achieve a specific function. \n\nHe believes that RAG is easier to onboard and
use compared to fine-tuning, making it a more accessible choice for most people.
However, he also acknowledges that fine-tuning could potentially take over some
aspects of what RAG does in the future.", additional_kwargs={})]
```

Here you can see that with that follow up question, it retains the context we have so far.

```
query_2 = "How does he think that it will evolve over time?"
response_2 = chat_eng.chat(query_2)
```

```
print(response_2.response)
```

```
Jerry believes that there will be an increasing interplay between fine-tuning
and RAG as time goes on. He thinks that fine-tuning will probably memorize some
high-level concepts of knowledge, while RAG will be there to supplement aspects
that it doesn't know. He also suggests that there might be some aspect of
personalization of memory in the future, where a personalized assistant can
learn behaviors over time and learn through conversation history. He believes
that this will be part of the ongoing continuous fine-tuning.
```

In production, we'll want to save the chat history so that we can tune and debug our model over time but for now we can just look at this particular memory.

Memory (or rather, how we store and compress it) is another thing we can tune and there's a lot of discussion about "memory" in industry.

If you think about it, storing information in a semantic database (e.g., the retrieval part of RAG), is a kind of "memory".

The model itself has its own "memory" from all the data that it's seen.

In fact, the authors of the RAG paper called this out explicitly in their blog post:

> *RAG's true strength lies in its flexibility. Changing what a pretrained language model knows entails retraining the entire model with new documents. With RAG, we control what it knows simply by swapping out the documents it uses for knowledge retrieval.*

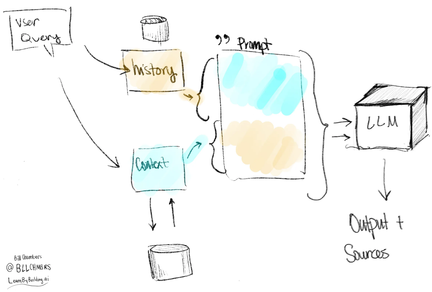
Memory and concepts of memory will evolve over the coming years, but for now what seems most practical is storing facts and references in a semantic database and letting the model / managed conversation memory handle the rest.

So what's happening under the hood?

Alright, so we've built this application, let's talk about what's going on under the hood.

The API we're using above is the "high level" API. It's intended to simplify the base use cases. In more detail, the flow we see when querying the model is:

1. the `chat_engine` receives a message
2. It gets the context that it wants to give to the model (e.g., the sources). It includes those with a prompt (we've just used the basic prompt at this point).
3. It gets the history of the conversation
4. It passes all of that into the LLM



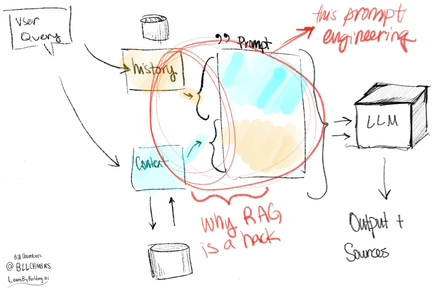
In short, llamaindex adds the message to the memory, fetches the context (from the database / retriever), then passes all of that into the LLM. Now all if this can be customized. We'll cover that information in later topics.

> 🚨🚨🚨 *Advanced readers only* 🚨🚨🚨 *For those that want to dive super deep, take a look at the `chat_engine` code. The code is quite neat, so it's not hard to understand. One word of caution, don't depend on internal names in production (or even development). The LlamaIndex authors are deliberate about what's in public and what's in private. However, this will give you a better understanding of what exactly is happening under the hood.*

so, why is RAG a hack?

The podcast title, the discussion. RAG is a hack, let's talk about why that is.

At its core, RAG involves stuffing the right information into the prompt. It's no different from "prompt engineering" that you might be doing on your local machine - it just does that with an automated database lookup.



That makes it "inelegant" because it's just forcing the right stuff into the prompt to get better answers. That makes it very practical and effective, but not necessarily perfect and not necessarily the most "clean" solution either.

But if it works, it works and that's why it has so much excitement.

Final thoughts

We've learned a ton in this post about building RAG applications, from basic querying to chatbots. As we discussed in our previous post, there's much to tune. In later posts, we'll discuss the details of how we can go about customizing our prompts, tuning the chunk sizes for our context and so much more!

This post was originally published on LearnByBuilding.AI. *See the notebook and code on Github.* Please reach out to me on twitter with any questions or feedback!

LEARN BY BUILDING AI

I help people understand AI concepts through practical implementation and hands-on learning.

Quick Links

About
Newsletter
Contact

Social Links

Github
Twitter
Linkedin
Youtube

Subscribe Newsletter

Your Email Address

Subscribe Now

Copyright Bill Chambers